

Funções Parte I

Definição: Função é um grupo de instruções destinado ao processamento de uma certa rotina, ou seja, um processo que manipula compara e trata valores entre outras rotinas.

anatomia de uma função:

“tipo_de_retorno NomeDaFuncao (Parâmetros) {Processamento}”

- tipo_de_retorno: int, float, char, bool, e o caso especial, que é o void, quando não queremos retornar nada da função.

-NomeDaFuncao: Como vamos chamar a função após as definição e estruturação

-Parâmetros: cada um precisa de um tipo e um nome

Exemplo em C:

```
int calcular(int a, int b) {  
    int retorno;  
    retorno = a + b;  
    return retorno;  
}
```

Tipo do retorno = “int” (inteiro)

NomeDaFuncao = “calcular”

Parâmetros = “int a”, “int b” (tipo e nome)

Processo = pegar dois inteiros somar entre si e devolver a soma dos dois parâmetros.

Ciclo de vida de uma variável e seus escopos dentro do código e dentro das funções

Geralmente cada agrupamento de códigos possui um escopo de variáveis, sendo assim, o escopo da função main possui algumas variáveis, o escopo da função fora da main possui outras variáveis e o escopo de uma classe possui outras

Mas podemos subdividir essas variáveis em 3 tamanhos

S = Variável global de sistema(perigosa)

A = Variável local, variável que pode sofrer alterações, mas apenas no escopo em que foi definida.

R = Passa por um processo de cópia, essa é a ponte direta com o que já estudamos: é exatamente a passagem por valor.

Podemos observar a variável de parâmetro “R” no seguinte exemplo:

```
void dobrar(int x) {  
    x = x * 2;  
    printf("Dentro da funcao, x = %d\n", x);  
}  
  
int main() {  
    int x = 5;  
    printf("Antes da chamada, x = %d\n", x);  
    dobrar(x);  
    printf("Depois da chamada, x = %d\n", x);  
}
```

//Saída:

Antes da chamada, x = 5

Dentro da funcao, x = 10

Depois da chamada, x = 5

Quando uma variável é passada por parâmetro ela é atribuída na memória com um espaço diferente, esses valores podem coincidentemente possuírem a mesma nomenclatura porem, quando esse endereço é passado por parâmetro para a função e ele não é um objeto a memória cria um novo endereço de memória para aquele valor, resumidamente uma “cópia”. Por conta disso o valor que é enviado $x = 5$ dentro da função se torna 10, mas a variável que está no “Main” não sofreu alteração e ainda possui o valor de 5.

Teste de mesa (a ferramenta de raciocínio)

Passo	Instrução	x (do main)	x (da função)
1	$x = 5$	5	não existe
2	<code>dobrar(x)</code> → cria a cópia	5	5
3	$x = x * 2$	5	10
4	fim da função → cópia destruída	5	não existe